

实验 3 生成对抗网络 GAN 及 DCGAN 实验

一、 实验目的

学习掌握生成对抗网络 (GAN) 和深度卷积生成对抗网络 (DCGAN) 和 BIGGAN 基本原理; 利用 DCGAN 和 BIGGAN 进行图片生成。

二、 实验内容

熟悉并利用 DCGAN 算法进行 MNIST 数字手写体图像生成。在此基础上, 通过查找资料, 分析 BIGGAN 网络, 分别使用 DCGAN 和 BIGGAN 两种网络进行图像生成和对比。

三、 实验环境

硬件: x86_64 Centos 3.10.0 服务器/GPU 服务器、GPU、PC 上位

机; 软件: torch1.0.1、python3.5。

四、 实验原理

生成式对抗网络 (GAN, Generative Adversarial Networks) 是一种深度学习模型, 是近年来复杂分布上无监督学习最具前景的方法之一。模型通过框架中 (至少) 两个模块: 生成模型 (Generative Model) 和判别模型 (Discriminative Model) 的互相博弈学习产生相当好的输出。原始 GAN 理论中, 并不要求 G 和 D 都是神经网络, 只需要是能拟合相应生成和判别的函数即可。但实用中一般均使用深度神经网络作为 G 和 D。

DCGAN, Deep Convolutional Generative Adversarial Networks 是生成对抗网络 (Generative Adversarial Networks) 的一种延伸, 将卷积网络引入到生成式模型当中来做无监督的训练, 利用卷积网络强大的特征提取能力来提高生成网络的学习效果。

DCGAN 算法特点:

1. 在判别器模型中使用 strided convolutions 来替代空间池化 (pooling), 而在生成器模型中使用 fractional strided convolutions, 即 deconv, 反卷积层。
2. 除了生成器模型的输出层和判别器模型的输入层, 在网络其它层上都使用了 Batch Normalization, 使用 BN 可以稳定学习, 有助于处理初始化不良导致的训练问题。
3. 去除了全连接层, 而直接使用卷积层连接生成器和判别器的输入

层以及输出层。

4. 在生成器的输出层使用 Tanh 激活函数，而在其它层使用 ReLU；在判别器上使用 leaky ReLU。

五、实验分析及步骤

1、准备数据集。本次实验使用 DCGAN 算法进行 MNIST 数字手写体图像生成，实验前需要下载 MNIST 数据集，load_dataset(batch_size=10, download=True) 函数用于加载数据集，并划分为训练集和测试集，其中 download 为 true 时，可在线下载 MNIST 数据集，若本地已有 MNIST 数据集，则 download 可设为 False。

```
1. def load_dataset(batch_size=10, download=True):
2.     transform = transforms.Compose([transforms.ToTensor(),
3.                                     transforms.Normalize([0.5],
4.                                                         [0.5])])
5.     trainset = torchvision.datasets.MNIST(root='../data', train=True,
6.                                           download=download,
7.                                           transform=transform)
8.     trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
9.                                               shuffle=True, num_workers=2)
10.
11.     testset = torchvision.datasets.MNIST(root='../data', train=False,
12.                                          download=download,
13.                                          transform=transform)
14.     testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size,
15.                                              shuffle=False, num_workers=2)
16.
17.     return trainloader, testloader
```

2、模型定义。本实验使用 DCGAN 算法，包含两个模型，分别为判别器 DCGAN_Discriminator 和生成器 DCGAN_Generator。

判别器：首先判别器使用卷积操作，激活函数使用 leaky_relu 函数，每一个 layer 都需要使用 batch normalization。Pytorch 的 leaky_relu 使用 torch.nn.functional.leaky_relu(input, negative_slope=0.01, inplace=False) 实现，batch normalization 使用 torch.nn.BatchNorm2d 实现。模型前三层都是 conv2d+BN+leaky_relu，最后一层 conv2d+sigmoid 操作。


```

39.         self.BN3 = nn.BatchNorm2d(32)
40.         self.conv4 = nn.ConvTranspose2d(32, n_channel, 3,
41.                                         stride=2, padding=2, bias=False)
42.
43.     def forward(self, x):
44.         """
45.         补充代码
46.         """
47.         return x

```

3、训练模型。

函数 `gen_noise(n_instance, n_dim=2)` 用于随机生成生成器的输入，返回值为从标准正态分布中抽取的一组随机数。同学们在实验过程中也可以尝试生成其他分布。其中 `n_instance` 为生成的输入个数，应该与 `batch_size` 保持一致，`n_dim` 为生成的维度，与生成器模型的 `noise_dim` 保持一致。

```

1. def gen_noise(n_instance, n_dim=2):
2.     return torch.randn(n_instance, n_dim, 1, 1)

```

训练过程。函数

```

1. train_DCGAN(Dis_model, Gen_model, D_criterion, G_criterion, D_optimizer,
2.             G_optimizer, trainloader, n_epoch, batch_size, noise_dim,
3.             n_update_dis=1, n_update_gen=1, use_gpu=False, print_every=1
4.             0,
5.             update_max=None)

```

用于训练模型。其中 `Dis_model` 和 `Gen_model` 为判别器和生成器模型；`D_criterion` 和 `G_criterion` 为判别器和生成器的损失函数；`D_optimizer` 和 `G_optimizer` 为判别器和生成器的优化函数；参数 `n_update_dis` 和 `n_update_gen` 指明训练过程中更新模型参数的频率，取值为整数；`use_gpu` 参数指明训练过程中是否使用 GPU，若为 `True` 则使用 GPU；`update_max` 指明在每个 epoch 的训练过程中训练的最大次数。因此训练过程如下：

```

1. def train_DCGAN(Dis_model, Gen_model, D_criterion, G_criterion, D_optimizer,
2.                 G_optimizer, trainloader, n_epoch, batch_size, noise_dim,

```

```

3.             n_update_dis=1, n_update_gen=1, use_gpu=False, print_every=1
   0,
4.             update_max=None):
5.     for epoch in range(n_epoch):
6.
7.         D_running_loss = 0.0
8.         G_running_loss = 0.0
9.
10.        for i, data in enumerate(trainloader, 0):
11.            # get the inputs from true distribution
12.            true_inputs, _ = data
13.            if use_gpu:
14.                true_inputs = true_inputs.cuda()
15.                true_inputs = Variable(true_inputs)
16.
17.            # get the inputs from the generator
18.            noises = gen_noise(batch_size, n_dim=noise_dim)
19.            if use_gpu:
20.                noises = noises.cuda()
21.            fake_inputs = Gen_model(Variable(noises))
22.            # 保存训练过程中生成的图片，在自己电脑上实验的同学可以保存训练过程中生
   成的图片查看
23.            #fake_image_name = str(epoch)+"_"+str(i)+'.png'
24.            #image_path = os.path.join("../data/gen", fake_image_name)
25.            #if i%100 == 0:
26.                #torchvision.utils.save_image(fake_inputs, nrow=10, pad_valu
   e=255, fp=image_path)
27.            inputs = torch.cat([true_inputs, fake_inputs])
28.            # get the labels
29.            labels = np.zeros(2 * batch_size)
30.            labels[:batch_size] = 1
31.            labels = torch.from_numpy(labels.astype(np.float32))
32.            if use_gpu:
33.                labels = labels.cuda()
34.                labels = Variable(labels)
35.
36.            # Discriminator
37.            D_optimizer.zero_grad()
38.            outputs = Dis_model(inputs)
39.            D_loss = D_criterion(outputs[:, 0], labels)
40.            if i % n_update_dis == 0:
41.                D_loss.backward(retain_graph=True)
42.                D_optimizer.step()
43.

```

```

44.         # Generator
45.         if i % n_update_gen == 0:
46.             G_optimizer.zero_grad()
47.             G_loss = G_criterion(outputs[batch_size:, 0],
48.                                 labels[:batch_size])
49.             G_loss.backward()
50.             G_optimizer.step()
51.
52.         # print statistics
53.         D_running_loss += D_loss.item()
54.         G_running_loss += G_loss.item()
55.         if i % print_every == (print_every - 1):
56.             print('[%d, %5d] D loss: %.3f ; G loss: %.3f' %
57.                   (epoch+1, i+1, D_running_loss / print_every,
58.                    G_running_loss / print_every))
59.             D_running_loss = 0.0
60.             G_running_loss = 0.0
61.
62.         if update_max and i > update_max:
63.             break
64.
65.     print('Finished Training')

```

4、训练结果。生成器和判别器学习率均设为 $4e-4$ ，batch_size 设为 100，判别器和生成器的损失函数均使用交叉熵损失函数 `torch.nn.BCELoss()`，BCELoss 的作用是计算二分类 target 和 output 之间的 loss。优化器均使用 `torch.optim.Adam()`。其他参数设置如下，即：

```

1. def run_DCGAN(n_epoch=50, batch_size=100, use_gpu=False, dis_lr=4e-4,
2.               gen_lr=4e-4, n_update_dis=1, n_update_gen=1, noise_dim=100,
3.               n_channel=1, update_max=None):
4.     # loading data
5.     trainloader, testloader = load_dataset(batch_size=batch_size, download=False)
6.
7.     # initialize models
8.     Dis_model = DCGAN_Discriminator(n_channel=n_channel)
9.     Gen_model = DCGAN_Generator(n_channel=n_channel, noise_dim=noise_dim)
10.
11.     if use_gpu:
12.         Dis_model = Dis_model.cuda()
13.         Gen_model = Gen_model.cuda()
14.

```

```

15.     # assign loss function and optimizer (Adam) to D and G
16.     D_criterion = torch.nn.BCELoss()
17.     D_optimizer = optim.Adam(Dis_model.parameters(), lr=dis_lr,
18.                               betas=(0.5, 0.999))
19.
20.     G_criterion = torch.nn.BCELoss()
21.     G_optimizer = optim.Adam(Gen_model.parameters(), lr=gen_lr,
22.                               betas=(0.5, 0.999))
23.
24.     train_DCGAN(Dis_model, Gen_model, D_criterion, G_criterion, D_optimizer,
25.                 G_optimizer, trainloader, n_epoch, batch_size, noise_dim,
26.                 n_update_dis, n_update_gen, update_max=update_max)

```

可得结果如下图所示，图 1 为第 3 个 epoch 训练 200 次得到的结果；图 2 位为第 11 个 epoch，训练 300 次的结果。

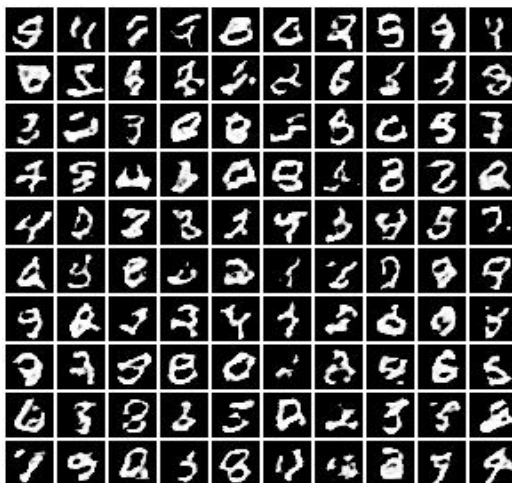


图 1



图 2

六、 扩展实验

1) 进一步分析代码，理解 DCGAN 结构，特别是熟悉构建生成网络和判别网络的过程。用自己感兴趣的数据集（比如汽车、飞机或者人物等）进行训练和测试。

2) GAN 变体 BIGGAN 在图片生成方面有巨大的进展，可生成几乎以假乱真的图片。分析和使用 BIGGAN (<https://github.com/AaronLeong/BigGAN-pytorch>) 对扩展实验 1) 的图像进行训练和测试，并对两个算法结果进行对比分析。